

Yaad — 24h Build Plan (3 humans + 3 supervised agents)

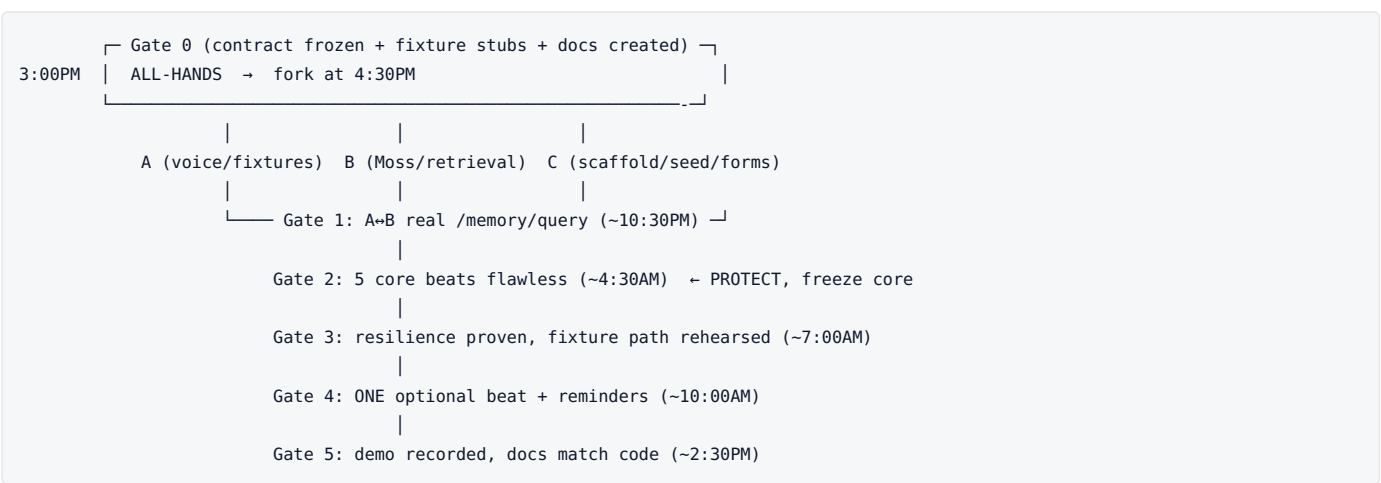
Window: 3:00 PM Day 1 → 3:00 PM Day 2 (24h). **Team / tracks:** Keshav → **Track A Voice Agent** · Rushil → **Track B Memory Engine** · Raghav → **Track C Caregiver Web**. **Operating model:** each owner runs one supervised Claude Code agent in their package only (no two agents in one module). Humans own hard logic + integration; agents do scaffolding/boilerplate/tests and keep their [CLAUDE.md](#) / [STATUS.md](#) current.

Drop this in the repo as [PLAN.md](#). It sits *next* to the spec ([CLAUDE.md](#)), the contract ([CONTRACT.md](#)), and the live log ([STATUS.md](#)). It does not replace them.

The 5 rules that govern this whole plan

- Moss is the hero.** Every spoken answer is grounded in retrieved memory. The signature live beat is **add-fact-live**. Protect it above all polish.
- Freeze the contract by 4:30 PM (Gate 0).** After that, [CONTRACT.md](#) changes are rare + LOUD (all-hands, regenerate types, bump version, post in [STATUS.md](#)).
- Fixtures are the decoupler.** Everyone codes against fixture stubs first so nobody is blocked. Real wiring happens only at named gates.
- Core (\$★) flawless before any optional beat.** who-is-this · pills-today · add-fact-live · wifi-off · Hindi. Vision/wander/extra polish come *after* these five never-fail.
- Living docs are part of "done."** A change isn't done until its [CLAUDE.md](#) / [CONTRACT.md](#) / [STATUS.md](#) is updated *in the same commit*. Session bookends: re-read before, update after.

Critical path & dependency map (why this parallelizes)



The only hard blocker is Gate 0. Until the contract is frozen and [memory-engine](#) returns *something* (even fixtures), A and C are stuck. So the first 90 minutes are the most valuable of the whole hackathon — do them together, in one room, before anyone forks.

Phase 0 — ALL-HANDS KICKOFF · 3:00 PM → 4:30 PM (90 min)

Nobody touches their own track yet. You build the shared spine together.

Owner	Task	Output
Rushil (lead)	Author memory-engine/app/schemas.py (Pydantic) = the \$3 data model + endpoint shapes. Export contract.openapi.yaml .	Frozen schema
Raghav	Run openapi-typescript → caregiver-web/lib/types.ts . Confirm drift fails the web build. Scaffold the three packages/*/CLAUDE.md + root STATUS.md .	Type pipeline + docs skeleton
Keshav	Stand up fixtures/ skeleton for all 5 beats (who-is-this, pills-today, add-fact-live, wifi-off, Hindi) — even hand-written JSON now. Write CONTRACT.md from schemas.py .	Fixtures + contract doc
Rushil	Stub memory-engine/app/main.py (FastAPI) returning fixture payloads for /memory/query , /memory/temporal , /memory/write , /reminders/duo , /health .	Stubbed engine on :8000

Parallel verification errands (do during the same 90 min, don't wait for office hours later): - **Rushil → Moss [CONFIRM]:** on-device/WASM vs cloud, instant upsert latency, cross-lingual embeddings, exact SDK calls. *This validates the entire hero story — do it first.* If on-device is shaky → fall back to Moss-cloud and reframe "wifi-off" as "edge-ready" in the pitch. - **Keshav → MiniMax [CONFIRM]:** Hindi voice id, streaming TTS, group id. Also Deepgram + LiveKit/Pipecat exact calls + TrueFoundry base_url/model. - **Raghav → keys/env:** fill `.env` (Supabase, LiveKit, Deepgram, MiniMax, TrueFoundry, Twilio, Unsiloed). `docker compose up` boots all three services empty.

Gate 0 exit (4:30 PM) — do NOT fork until all true: - ☐ `CONTRACT.md` frozen, version line set, posted in `STATUS.md`. - ☐ `memory-engine` answers all core endpoints with fixtures (`curl` passes). - ☐ `types.ts` generated from OpenAPI; web build green. - ☐ All 4 living docs exist (root `CLAUDE.md` , 3 package `CLAUDE.md` , `CONTRACT.md` , `STATUS.md`). - ☐ Moss instant-update confirmed (or fallback decided + logged).

Per-track playbooks (run in parallel from 4:30 PM)

TRACK A — Voice Agent · Keshav (you)

Goal: warm, low-latency, barge-in voice loop where every answer is grounded. **Pipeline:** LiveKit → VAD → Deepgram STT → intent/lang → `memory_client.query()/temporal()` (speculative on partial transcript) → LLM (TrueFoundry) w/ grounding prompt → MiniMax TTS → playback; barge-in on VAD.

Phase	You (human)	Your agent	Files	Done when
A0	Wire LiveKit + Pipecat transport; VAD.	Scaffold <code>agent.py</code> , <code>transports.py</code> , <code>memory_client.py</code> against fixtures.	<code>agent</code> , <code>transports[CONFIRM]</code> , <code>memory_client</code>	Pipeline runs end-to-end on stubs
A1	Deepgram STT + MiniMax echo loop; confirm Hindi voice live.	Boilerplate <code>stt_deepgram.py</code> , <code>tts_minimax.py</code> + retry/stream handling.	<code>stt_deepgram[CONFIRM]</code> , <code>tts_minimax[CONFIRM]</code>	Speak in → transcribed → spoken back, EN + HI
A2	Real <code>/memory/query</code> + grounding system prompt + barge-in.	<code>llm.py</code> TrueFoundry client + prompt assembly.	<code>llm[CONFIRM]</code>	"who is Leo?" warm + grounded
A3	Latency pass: speculative fire on partial transcript + streaming TTS → <~1s. Build the latency overlay (live/cached + ms).	Instrument timing, overlay component.	<code>overlay</code>	p95 voice <~1s
A4	<code>fallback.py</code> : 3s timeout on any memory/vision/TTS call → serve matching fixture + cached TTS; header live/cached indicator.	Wire fixtures into fallback, cache TTS clips.	<code>fallback</code>	Kill memory-engine → 5 beats still answer
A5	Proactive reminders: scheduler polls <code>/reminders/due</code> → same grounded TTS path.	<code>reminders_client.py</code> .	<code>reminders_client</code>	Agent says "8 — white heart pill; Sarah at 3"

Grounding system prompt (freeze this verbatim): "You are Yaad, a warm companion for someone with memory loss. State ONLY facts in the provided MEMORY context. If empty/low-confidence, say you're not sure and offer to check with the family. Never invent people/events/dates. Short, calm, warm. Match the user's language (English/Hindi/Hinglish)."

TRACK B — Memory Engine · Rushil (the technical hero)

Goal: turn Moss retrieval into a grounded, temporal, traversable graph behind the \$3 API.

Phase	Rushil (human)	His agent	Files	Done when
B1	<code>moss_client.py</code> index/query/instant upsert [CONFIRM on-device]; <code>/memory/write</code> .	DB layer, Supabase wiring.	<code>moss_client[CONFIRM]</code> , <code>db</code>	A <code>/memory/write</code> retrievable <1s
B2	<code>graph.py</code> entities/episodes/edges + 1-hop traversal; <code>retrieval.py</code> <code>score = $\alpha \cdot \text{sem} + \beta \cdot \text{recency} + \gamma \cdot \text{salience} + \delta \cdot \text{graph_prox}$</code> , <code>recency = exp(-$\lambda \cdot \Delta t$)</code> .	Scoring scaffold + unit tests.	<code>graph</code> , <code>retrieval</code>	Query returns ranked items w/ provenance
B3	Grounding: confidence gate τ → safe refusal; provenance on every item.	Threshold plumbing, refusal drafts.	<code>grounding</code>	Ungrounded → safe refusal, never confabulates
B4	Temporal: time-intent routing; "pills today" → today's <code>med_log</code> ; "is X coming" → upcoming events.	<code>temporal.py</code> routing skeleton.	<code>temporal</code>	"pills today" reflects a just-logged dose
B5	<code>/memory/capture</code> (extract on explicit "remember this..." + review),	Endpoint	<code>capture</code> , <code>reminders</code> ,	Timeline +

	<code>/memory/timeline</code> , <code>/reminders/due</code> , <code>location.py</code> , (opt) <code>vision.py</code> .	scaffolds + fixtures.	<code>location</code> , <code>vision</code>	reminders return real data
B6	<code>smoke_test.py</code> + ~20-case grounding/latency table for the demo.	Generate test cases + table.	<code>tests/</code>	p95 <code>/memory/query</code> <60ms server-side

TRACK C — Caregiver Web · Raghav

Goal: the family's window — and the engine of the add-fact-live beat.

Phase	Raghav (human)	His agent	Files	Done when
C0	Next.js scaffold; typed <code>api.ts</code> from <code>types.ts</code> .	Project boilerplate, routing.	<code>lib/api.ts</code> , <code>lib/types.ts</code>	Web build green, calls engine
C1	Add-memory forms (person/event/med/story) → <code>/memory/write</code> . One-click fast — this <i>is</i> add-fact-live.	Form components + validation.	<code>app/memories</code>	"Leo's birthday is Saturday" → answerable in seconds
C2	<code>seed_amma.py</code> — rich, real-feeling Amma (84; grandson Leo, daughter Sarah, meds, routine, episodes, home + park).	Seed data generation.	<code>scripts/seed_amma.py</code>	One command seeds a believable life
C3	Graph view (<code>MemoryGraph.tsx</code> , react-force-graph) + timeline (<code>/memory/timeline</code>).	Component scaffolds.	<code>app/graph</code> , <code>app/timeline</code>	Graph + timeline render
C4	Care dashboard: "topics to reinforce with her." NO fake clinical/health score.	Dashboard layout.	<code>app/(dashboard)</code>	Caregiver guidance, no fake metrics
C5	Safety view: set home + safe-zone on map; ordered contacts; live location + alert history.	Map + form scaffolds.	<code>app/safety</code>	Geofence settable
C6	<code>docs/ARCHITECTURE.md</code> diagram — Moss at the center — for the pitch.	Diagram draft.	<code>docs/ARCHITECTURE.md</code>	Pitch-ready system diagram

Hour-by-hour master timeline

Clock	Keshav (A)	Rushil (B)	Raghav (C)	Sync
3:00–4:30 PM	Fixtures + <code>CONTRACT.md</code>	schemas + OpenAPI + Moss CONFIRM	<code>types.ts</code> + docs + env	ALL-HANDS
4:30–7:30 PM	A0 pipeline, A1 echo loop	B1 Moss client + <code>/memory/write</code>	C0 scaffold + start C2 seed	—
~7:30 PM	—	—	—	Dinner / 15-min standup
7:30–10:30 PM	A2 real query + grounding + barge-in	B2 graph + retrieval, B3 grounding	C1 forms + finish C2 seed	—
~10:30 PM	—	—	—	GATE 1: A↔B "who is Leo?" live
10:30 PM–1:30 AM	A3 latency + overlay	B4 temporal + B5 reminders/due	C3 graph + timeline	—
1:30–4:30 AM	Core hardening — all hands on the 5 beats (rotate rest, see below)			GATE 2: 5 beats flawless → PROTECT
4:30–7:00 AM	A4 fallback + capture 5 fixtures	B6 eval harness + smoke test	C4 dashboard + C5 safety	GATE 3: kill engine, fixture path holds
7:00–10:00 AM	A5 reminders	support optional beat	C6 architecture diagram	GATE 4: ONE optional beat + reminder
10:00 AM–2:30 PM	Rehearse demo, latency overlay polish	Eval table on screen, final grounding check	UI polish, diagram final	Record 90s + rehearse fixture path
2:30–3:00 PM	Buffer / final docs sync (§16 checklist)			GATE 5: done

Overnight rest (1:30–6:00 AM): rotate so at least two people are awake and no agent runs unsupervised. Suggested: Raghav rests 1:30–3:30, Keshav 3:30–5:30, Rushil 5:30–7:30 (Rushil rests last so he's fresh for the eval table and any Moss firefighting in the morning). Adjust to who's flagging.

The five gates (these are go/no-go, not vibes)

- **Gate 0 (4:30 PM):** contract frozen, stubs answering, docs created. *No forking before this.*
- **Gate 1 (~10:30 PM):** voice agent gets a real grounded answer from the real engine — "who is Leo?" → "that's Leo, your grandson," warm, <~1s-ish. First proof the system is real.
- **Gate 2 (~4:30 AM): all five core beats flawless** end-to-end on the live path. After this gate, the core is *frozen* — touch it only to fix a regression. Everything new goes in branches.
- **Gate 3 (~7:00 AM):** resilience proven. Kill `memory-engine`; all 5 beats still answer from fixtures with the live/cached header flipping correctly. Rehearse this path now and once more before recording.
- **Gate 4 (~10:00 AM):** exactly **one** optional beat in + one reminder beat. Do not start a second optional beat.
- **Gate 5 (~2:30 PM):** 90s video recorded, fixture path rehearsed, all Claude files match shipped code (§16).

Which optional beat? (decide AT Gate 3, not before)

Pick **one**. Recommendation: **Vision** ("point camera → that's Leo, your grandson"). - It reinforces the "memory is beautiful" frame; wander leans toward "look how sad this disease is," which the pitch explicitly avoids. - Fixture fallback is trivial: one pre-registered photo → one person ref → `answer_draft`. - It's a single-shot, demoable-in-a-noisy-room wow.

Choose **Wander** instead only if vision recognition is unstable by Gate 3 *and* the safety map/geofence is already solid. Wander needs map + geofence + Twilio + a believable live "I left the zone" moment, which is harder to stage convincingly. Whichever you pick: **reassure + keep-in-place + alert a human — never navigate**. Hard guardrail in code and in the pitch.

Running the three agents without chaos

Give each agent this kickoff line (paste into its first message), then supervise:

"You are the agent for `packages/<track>`. Before you code: re-read this package's `CLAUDE.md` + root `STATUS.md`. Code only against the frozen `CONTRACT.md` — never invent endpoint signatures; mark anything unverified `[CONFIRM]`. You do scaffolding, boilerplate, and tests; the human owns hard logic and integration. Anything you fake (fixture, hardcoded value, mocked endpoint) goes in `STATUS.md` under 'Faked / TODO real'. After you code: update this `CLAUDE.md` + `STATUS.md` in the same commit. Treat the docs as part of the deliverable."

Rules of engagement: - **One agent per package, ever**. No two agents in one module → no merge wars. - **Agents never touch `CONTRACT.md`**. Only a human, only at all-hands, only with a version bump + `STATUS.md` post. - **No silent stubs**. If it's faked for the demo, it's in `STATUS.md`. - **Session bookends are mandatory** for humans and agents alike — re-read docs in, update docs out.

Demo (last ~4.5h) — §15 script, ~90s

Why (grandmother, restrained) → "who is this?" → "did I take my pills today?" → **add-fact-live** → wifi-off → reveal ("a memory graph on Moss — search engines organize information; we organize a human life") + one Hindi line → close ("we gave a family their memories back") + sponsor thanks (Moss, MiniMax, LiveKit, Unsiloed).

- Record on the **live path**; rehearse the **fixture path** as backup. Have both ready at the podium.
- Keep the latency overlay and the ~20-case eval table visible during the technical reveal — they make "built on Moss" concrete.
- First-place levers: present Yaad as a **capability built on Moss**, not an app that uses it; land add-fact-live flawlessly; emotion→engineering in the same 90s; show don't tell; "a pattern any Moss dev could reuse."

Definition of done (§16 — final 30-min checklist)

- ☐ 5 beats live **and** from fixtures
- ☐ grounded-only verified (ungrounded → safe refusal, never confabulates)
- ☐ instant-update <1s · temporal "pills today" · wifi-off · Hindi exchange
- ☐ latency overlay + eval table
- ☐ one optional (vision **or** wander) + a reminder
- ☐ 90s video recorded + fixture path rehearsed
- ☐ all Claude files (root + 3 package `CLAUDE.md` + `CONTRACT.md` + `STATUS.md`) match shipped code
- ☐ keys rotation queued for post-event

Non-negotiable guardrails (§17)

Consent (family-managed, dignity-first) · grounded-only · confidence gate · provenance/auditable · augments-not-replaces · **NO fake clinical metrics** · **NEVER correct/distress the patient** · wander = reassure + alert human, never navigate · on-device privacy.